

Lập trình Java

Spring Data JPA

GV Nguyễn Đức Kiên



Nội dung

- 1) Giới thiệu spring data jpa
- 2) Cải tiến của Spring data jpa và JPA, Hibernate
- 3) Cách sử dụng Spring data JPA trong project
- 4) Các method được xây dựng sẵn của Spring data JPA
- 5) Query Creation
- 6) Các annotaion phổ biến
 1. @Query
 2. @Modifying
 3. @Transaction
- 7) native query trong spring data jpa

1. Giới thiệu Spring Data JPA

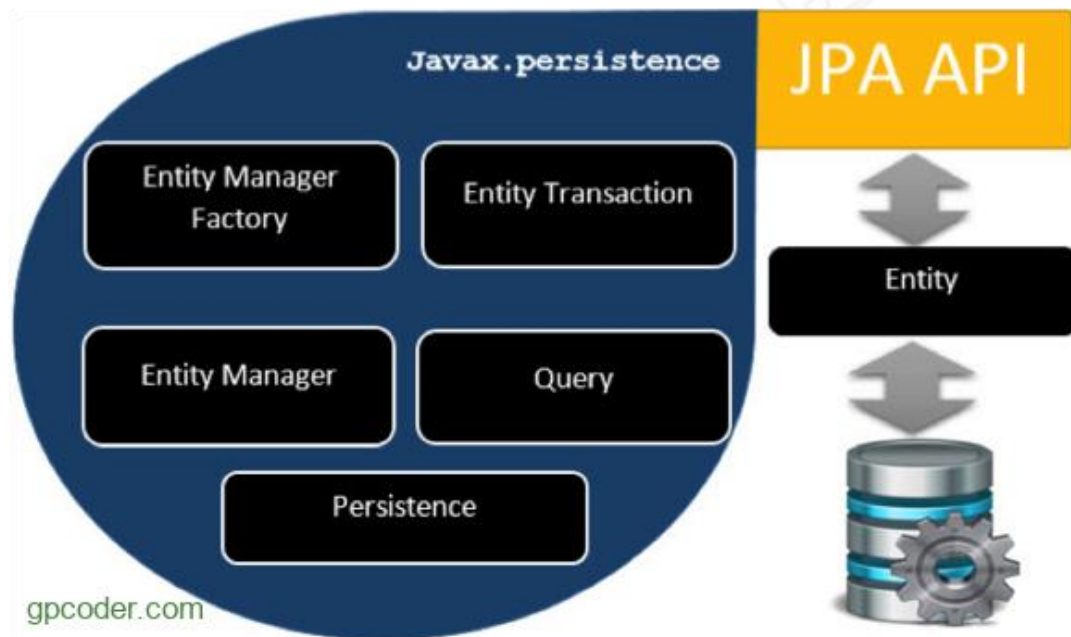
- Spring Data JPA là một phần mở rộng của Spring Data, giúp người phát triển dễ dàng thực hiện các thao tác cơ bản với cơ sở dữ liệu quan hệ thông qua Java Persistence API (JPA).
- Spring Data JPA hỗ trợ việc tương tác với cơ sở dữ liệu qua các Repository mà không cần viết quá nhiều mã thủ công. Điều này giúp tối ưu hóa mã nguồn, giảm thiểu sự phức tạp và tăng hiệu quả trong việc phát triển ứng dụng.
- JPA (Java Persistence API) là một API chuẩn của Java giúp thao tác với cơ sở dữ liệu quan hệ thông qua các đối tượng Java. Hibernate là một trong những framework ORM (Object-Relational Mapping) phổ biến, thực hiện JPA.
- Spring Data JPA giúp làm việc với Hibernate hoặc các framework ORM khác dễ dàng hơn, cung cấp thêm nhiều tính năng cao cấp như tự động tạo các phương thức truy vấn.

1.2) Tính năng chính của Spring Data JPA

- **CRUD tự động:** Spring Data JPA tự động tạo ra các phương thức CRUD từ tên của các phương thức trong repository.
- **Không cần phải viết câu lệnh SQL thủ công:** Thay vì phải viết câu lệnh SQL thủ công, bạn chỉ cần khai báo phương thức trong repository và Spring Data JPA sẽ tự động tạo câu truy vấn phù hợp.
- **Hỗ trợ truy vấn tùy chỉnh:** Có thể sử dụng `@Query` để viết các truy vấn SQL hoặc HQL (Hibernate Query Language) tùy chỉnh.
- **Hỗ trợ giao dịch(Transaction):** Spring Data JPA tích hợp tốt với Spring's `@Transactional`, giúp bạn dễ dàng quản lý các giao dịch.
- **Tự động hóa nhiều công việc:** Từ việc khởi tạo bảng trong cơ sở dữ liệu cho đến việc quản lý các thao tác CRUD, tất cả được Spring Data JPA tự động hóa, giúp tiết kiệm thời gian và công sức.

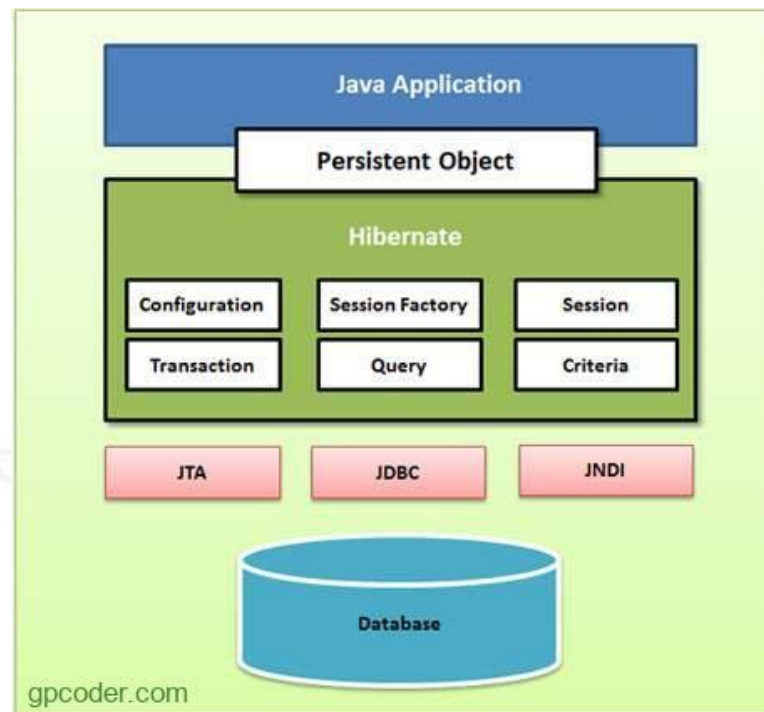
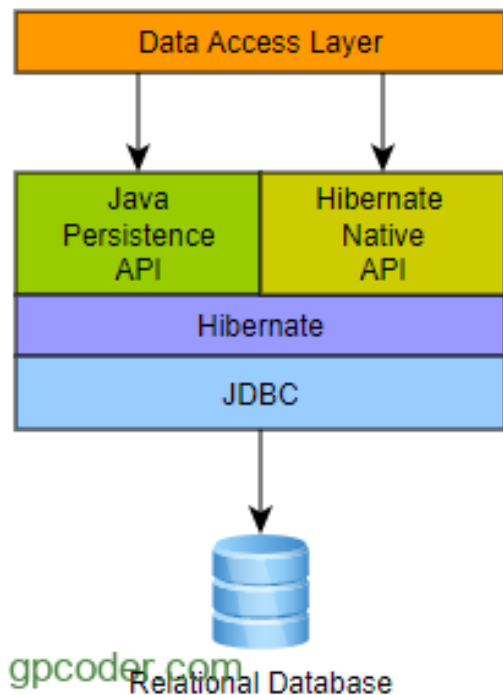
2. Cải tiến của Spring Data JPA và JPA, Hibernate

- **JPA** (Java Persistence API): JPA là một specification (đặc tả) chuẩn cho việc quản lý các thực thể (entities) và ánh xạ đối tượng (object-relational mapping - ORM) trong Java.
- **JPA** không phải là một framework cụ thể, mà là một API cho phép các framework ORM như Hibernate, EclipseLink, OpenJPA thực hiện các quy chuẩn này.
- **JPA** định nghĩa các phương thức cơ bản như ***persist()***, ***merge()***, ***find()***, ***remove()*** để thao tác với cơ sở dữ liệu mà không cần viết SQL thủ công.



2. Cải tiến của Spring Data JPA và JPA, Hibernate

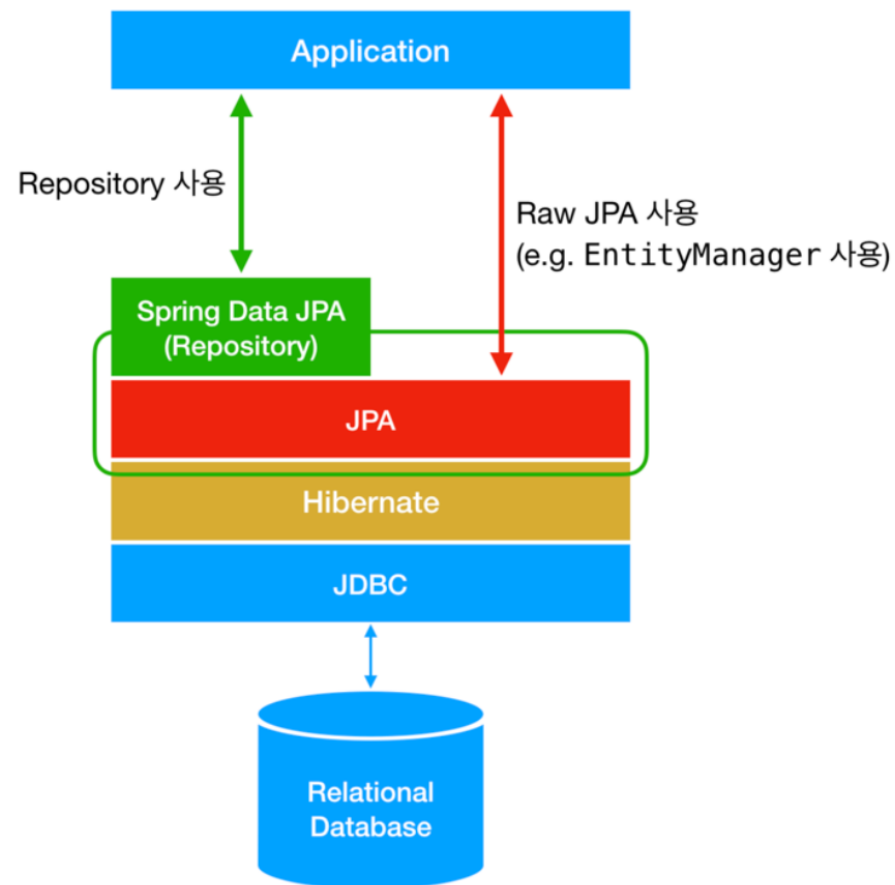
- **Hibernate:** Là một framework ORM phổ biến triển khai JPA. Hibernate giúp chuyển đổi giữa các đối tượng Java và các bảng cơ sở dữ liệu mà không cần phải viết SQL thủ công.



2. Cải tiến của Spring Data JPA và JPA, Hibernate

- **Spring Data JPA**

- Spring Data JPA là một thư viện của Spring Data, cung cấp một lớp trừu tượng cho JPA.
- Nó không phải là một phần của JPA hay Hibernate mà là một lớp bao bọc giúp giảm bớt mã nguồn cần viết khi làm việc với các thao tác dữ liệu trong ứng dụng Spring.
- Spring Data JPA giúp giảm thiểu sự phức tạp của việc tạo các repository cho các thao tác CRUD và truy vấn, tự động hóa quá trình tạo các truy vấn dựa trên tên phương thức, đồng thời cung cấp khả năng viết các truy vấn tùy chỉnh dễ dàng.



Cải tiến của Spring Data JPA và JPA, Hibernate

1) Giảm Bớt Boilerplate Code (Mã nguồn dư thừa):

- **JPA** yêu cầu bạn phải viết nhiều mã thủ công để tạo ra các repository, phương thức CRUD và các truy vấn.
- **Hibernate** cũng có sự tương tự, mặc dù nó giúp bạn thao tác với cơ sở dữ liệu mà không phải viết SQL thủ công, nhưng bạn vẫn phải tạo ra các repository và phương thức tương ứng.
- **Spring Data JPA** cải thiện vấn đề này bằng cách tự động tạo các repository cho bạn, chỉ cần khai báo interface kế thừa từ `JpaRepository` hoặc `CrudRepository` là bạn có thể sử dụng các phương thức CRUD mà không cần phải viết bất kỳ mã nào.

Cải tiến của Spring Data JPA và JPA, Hibernate

2) Tự Động Tạo Truy Vấn Từ Tên Phương Thức:

- Trong JPA và Hibernate, bạn cần phải viết các câu truy vấn hoặc sử dụng EntityManager để thực hiện các truy vấn.
- Spring Data JPA cho phép bạn chỉ cần đặt tên phương thức trong repository, và hệ thống sẽ tự động tạo ra câu truy vấn tương ứng.
- Ví dụ, findByCustomerId() sẽ được chuyển thành truy vấn SQL tự động.

3) Hỗ Trợ Truy Vấn Tùy Chỉnh Dễ Dàng:

- JPA cung cấp khả năng truy vấn với JPQL hoặc native SQL, nhưng bạn cần phải tự viết các câu truy vấn này trong mã.
- **Spring Data JPA** mở rộng khả năng này bằng cách hỗ trợ các câu truy vấn tùy chỉnh thông qua annotation @Query và hỗ trợ các phương thức truy vấn động mà không cần phải viết mã thủ công.

Cải tiến của Spring Data JPA và JPA, Hibernate

4) Phân Trang và Sắp Xếp Dễ Dàng:

- Trong JPA và Hibernate, việc phân trang và sắp xếp kết quả yêu cầu phải viết thêm logic trong các truy vấn hoặc sử dụng các API như Criteria API.
- Spring Data JPA cung cấp khả năng phân trang và sắp xếp trực tiếp thông qua Pageable và Sort, giúp việc này trở nên dễ dàng và tự động.

5) Tích Hợp Chặt Chẽ Với Spring Framework:

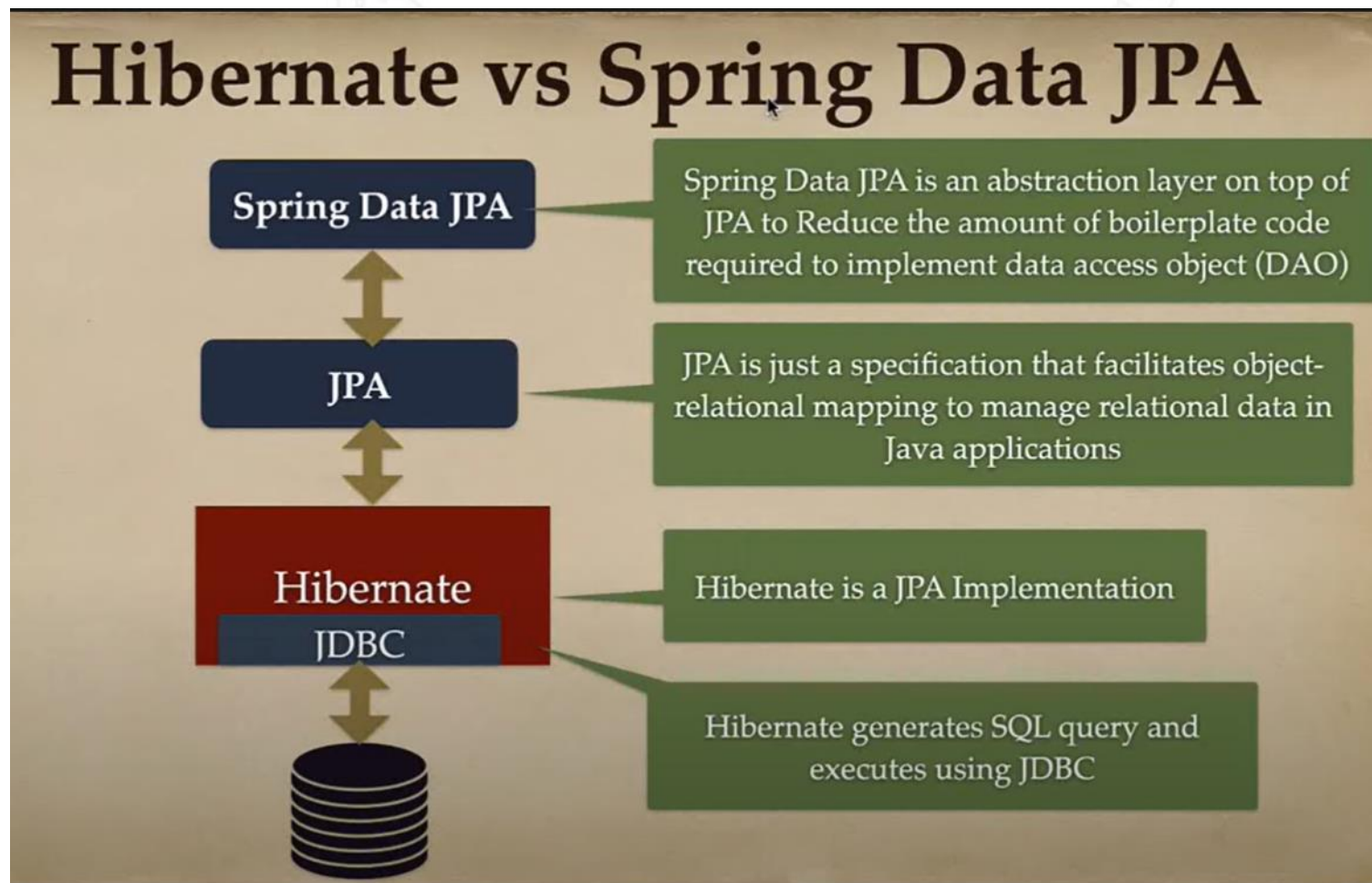
- Spring Data JPA tích hợp mạnh mẽ với Spring, giúp việc quản lý các repository và giao dịch trở nên đơn giản hơn.
- Chỉ cần khai báo một interface, và Spring sẽ tự động quản lý các repository, entity, và transaction. Spring Data JPA cũng có thể sử dụng các giao dịch được quản lý bởi Spring, ví dụ như sử dụng annotation @Transactional.

Cải tiến của Spring Data JPA và JPA, Hibernate

6) Tự Động Tạo Các Repository (Kho Dữ Liệu):

- Trong JPA và Hibernate, bạn phải tự viết mã cho repository để thực hiện các thao tác với cơ sở dữ liệu.
- Spring Data JPA tự động tạo ra các repository và cung cấp các phương thức CRUD sẵn có. Bạn chỉ cần khai báo một interface với tên phương thức thích hợp, và Spring Data JPA sẽ tạo ra mã cho bạn.

Cải tiến của Spring Data JPA và JPA, Hibernate



3) Cách sử dụng Spring Data JPA trong project

- Để sử dụng Spring Data JPA trong dự án Spring Boot, bạn thực hiện các bước sau:

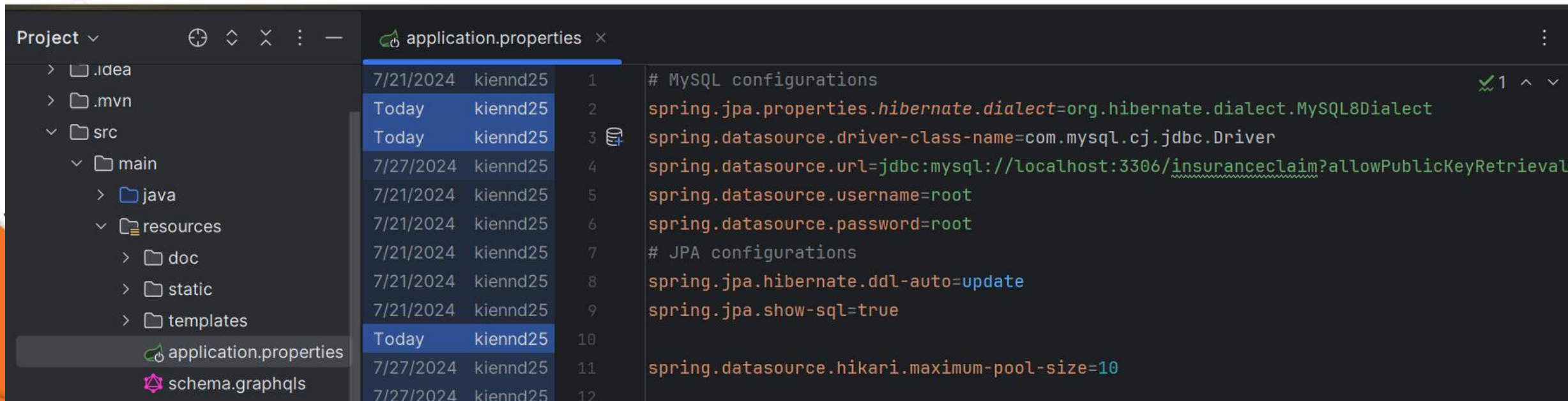
1) Thêm dependency vào pom.xml:

```
34     <dependency>
35         <groupId>org.springframework.boot</groupId>
36         <artifactId>spring-boot-starter-data-jpa</artifactId>
37     </dependency>
38     <dependency>
39         <groupId>com.mysql</groupId>
40         <artifactId>mysql-connector-j</artifactId>
41         <scope>runtime</scope>
42     </dependency>
```

3) Cách sử dụng Spring Data JPA trong project

2) Config các thông tin kết nối đến database trong file application.properties để tự động connect với database.

Lưu ý: Phải tự tạo database trước, hibernate chỉ hỗ trợ tự tạo ra table



The screenshot shows an IDE interface with a project explorer on the left and a code editor on the right. The project explorer shows a project structure with folders like .idea, .mvn, src, main, java, and resources. The code editor displays the application.properties file with the following content:

```
7/21/2024 10:25 1 # MySQL configurations
Today 10:25 2 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
Today 10:25 3 spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
7/27/2024 10:25 4 spring.datasource.url=jdbc:mysql://localhost:3306/insuranceclaim?allowPublicKeyRetrieval
7/21/2024 10:25 5 spring.datasource.username=root
7/21/2024 10:25 6 spring.datasource.password=root
7/21/2024 10:25 7 # JPA configurations
7/21/2024 10:25 8 spring.jpa.hibernate.ddl-auto=update
7/21/2024 10:25 9 spring.jpa.show-sql=true
Today 10:25 10
7/27/2024 10:25 11 spring.datasource.hikari.maximum-pool-size=10
7/27/2024 10:25 12
```

3) Cách sử dụng Spring Data JPA trong project

- 3) Tạo các class Entity để ánh xạ với các table trong database

The screenshot shows an IDE with a project named 'insurance-claim'. The left sidebar displays a file explorer with the following structure:

- src
 - main
 - java
 - com.t3h.insuranceclaim
 - config
 - controller
 - dto
 - entity
 - BaseEntity
 - ClaimDocumentEr
 - ClaimEntity**
 - ClaimStatusEntity
 - CustomerEntity
 - InsuranceProduct
 - ProductsEntity
 - RoleEntity
 - UserEntity
 - mapper
 - repository
 - security
 - service
 - utils

The right pane shows the code for 'ClaimEntity.java':

```
1 package com.t3h.insuranceclaim.entity;
2
3
4 import jakarta.persistence.*;
5 import lombok.Builder;
6 import lombok.Data;
7 import lombok.Getter;
8 import lombok.Setter;
9
10 import java.time.LocalDate;
11 import java.time.LocalDateTime;
12
13 @Entity 15 usages 1 kiennnd25
14 @Table(name = "claim")
15 @Getter
16 @Setter
17 public class ClaimEntity extends BaseEntity {
18
19     private String code;
20
21     @ManyToOne
22     @JoinColumn(name = "customer_id")
23     private CustomerEntity customerEntity;
24 }
```

3) Cách sử dụng Spring Data JPA trong project

- **B4) tạo 1 interface trong package repository để sử dụng spring data jpa**
- Cho extends từ JpaRepository để biến interface vừa tạo thành 1 repository được quản lý bởi spring data. Và sẽ tự động có các method được spring data jpa xây dựng sẵn

The screenshot shows an IDE with a project structure on the left and a code editor on the right. The project structure includes folders for `dto`, `entity`, `mapper`, `repository`, and `security`. The `repository` folder is expanded, showing several interfaces: `ClaimDocumentRepository`, `ClaimRepository` (selected), `ClaimStatusRepository`, `CustomerRepository`, `InsuranceProductRepository`, `ProductsRepository`, `RoleRepository`, and `UserRepository`. The code editor shows the content of `ClaimRepository.java`, which defines the interface and extends `JpaRepository`.

```
1 package com.t3h.insuranceclaim.repository;
2
3
4 import com.t3h.insuranceclaim.entity.ClaimEntity;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.stereotype.Repository;
7 @Repository 2 usages 1 kiennnd25 *
8 public interface ClaimRepository extends JpaRepository<ClaimEntity, Long> {
9
10 }
11
```


3) Cách sử dụng Spring Data JPA trong project

- B5) Sau bước 4 đã hoàn tất các bước để sử dụng được spring data jpa. B5 trở đi chỉ cần `@Autowired` vào các service hoặc các class muốn sử dụng.

```
@Service  @kiennd25 *
public class ClaimServiceImpl implements ClaimService {
    @Autowired
    private ClaimRepository claimRepository;

    public List<ClaimEntity> findAll(){ no usages new *
        List<ClaimEntity> claimEntities = claimRepository.findAll();
        return claimEntities;
    }
```

4) Các method được xây dựng sẵn của Spring data JPA

Spring data JPA đã cung cấp rất nhiều method có sẵn. Một số method nổi bật

1) save(S entity)

- Lưu hoặc cập nhật một đối tượng vào cơ sở dữ liệu. Nếu đối tượng đã tồn tại trong cơ sở dữ liệu (dựa trên ID), nó sẽ được cập nhật. Nếu không, nó sẽ được tạo mới.

```
@Service  @kiennd25 *
public class ClaimServiceImpl implements ClaimService {
    @Autowired
    private ClaimRepository claimRepository;
    @Autowired
    private ClaimMapper claimMapper;
    @Override new *
    public ClaimDTO create(ClaimDTO claimDTO) {
        // chuyển dữ liệu dạng DTO sang Entity để lưu vào database
        ClaimEntity claimEntity = claimMapper.toEntity(claimDTO);
        // sử dụng method save để lưu vào database
        claimEntity = claimRepository.save(claimEntity);
        return claimMapper.toDTO(claimEntity);
    }
}
```

4) Các method được xây dựng sẵn của Spring data JPA

2) findById(ID id)

- Tìm kiếm một đối tượng dựa trên ID. Phương thức này trả về một Optional, do đó bạn cần phải kiểm tra trước khi truy cập đối tượng.

```
@Override  @kiennnd25 *
public ClaimDTO getById(Long id) {
    ClaimEntity claimEntity = claimRepository.findById(id)
        .orElseThrow(
            // xử lý khi không tồn tại trong id trong database
            () -> new RuntimeException("Claim not found")
        );
    // convert kiểu entity sang DTO trả cho controller
    return claimMapper.toDTO(claimEntity);
}
```

4) Các method được xây dựng sẵn của Spring data JPA

3) findAll()

Lấy tất cả các đối tượng từ cơ sở dữ liệu dưới dạng danh sách (List).

```
0  @Service  @kiennd25 *
1  public class ClaimServiceImpl implements ClaimService {
2      @Autowired
3      private ClaimRepository claimRepository;
4      @Autowired
5      private ClaimMapper claimMapper;
6
7      public List<ClaimEntity> findAll(){ no usages new *
8          List<ClaimEntity> claimEntities = claimRepository.findAll();
9          return claimEntities;
10     }
```


4) Các method được xây dựng sẵn của Spring data JPA

4) count()

Đếm số lượng đối tượng trong bảng.

```
0  @Service  @kiennd25 *
1  public class ClaimServiceImpl implements ClaimService {
2      @Autowired
3      private ClaimRepository claimRepository;
4      @Autowired
5      private ClaimMapper claimMapper;
6
7      public Long countAllClaim(){ no usages new *
8          Long totalClaim = claimRepository.count();
9          return totalClaim;
10     }
```

4) Các method được xây dựng sẵn của Spring data JPA

5) deleteById(ID id)

Xóa một đối tượng theo ID.

```
@Service  @kiennd25 *
public class ClaimServiceImpl implements ClaimService {
    @Autowired
    private ClaimRepository claimRepository;
    @Autowired
    private ClaimMapper claimMapper;

    @Override  new *
    public void delete(Long id) {
        claimRepository.deleteById(id);
    }
}
```

4) Các method được xây dựng sẵn của Spring data JPA

6) delete(T entity)

- Xóa một đối tượng.

```
@Service  @kiennnd25 *
public class ClaimServiceImpl implements ClaimService {
    @Autowired
    private ClaimRepository claimRepository;
    @Autowired
    private ClaimMapper claimMapper;

    public void deleted(ClaimEntity claimEntity){ new *
        claimRepository.delete(claimEntity);
    }
}
```

5. Query Creation

- Query Creation trong Spring Data JPA cho phép bạn tự động tạo ra các truy vấn SQL hoặc JPQL dựa trên tên của phương thức trong interface repository mà không cần phải viết các câu truy vấn thủ công.
- Điều này giúp bạn tiết kiệm thời gian và làm giảm sự phức tạp khi thao tác với cơ sở dữ liệu.
- Spring Data JPA sử dụng quy tắc đặt tên phương thức (method name query derivation) để tự động tạo ra các truy vấn. Khi bạn tạo một phương thức trong repository, Spring Data JPA sẽ phân tích tên của phương thức và từ đó tạo ra truy vấn tương ứng.

5. Query Creation

Quy Tắc Đặt Tên Phương Thức (Method Name Query Derivation)

- Quy tắc này yêu cầu bạn phải tuân theo một cú pháp nhất định khi đặt tên cho phương thức, nhằm giúp Spring Data JPA tự động suy luận truy vấn SQL hoặc JPQL phù hợp.
- **Các phần của tên phương thức:**
 - ❑ Từ khóa tìm kiếm (search keyword): Các từ như find, read, get, count... được sử dụng để bắt đầu tên phương thức.
 - ❑ Tính từ mô tả các thuộc tính (attribute descriptors): Sau từ khóa tìm kiếm, bạn sử dụng tên các thuộc tính trong Entity để mô tả các trường cần truy vấn.
 - ❑ Toán tử logic: Bạn có thể sử dụng các từ như And, Or để kết hợp nhiều điều kiện trong truy vấn.

5. Query Creation

- **Quy tắc chung khi đặt tên:**

- **findBy:** Bắt đầu tên phương thức với findBy để thực hiện tìm kiếm. Phần còn lại của tên phương thức mô tả các thuộc tính của Entity.
- **And/Or:** Bạn có thể sử dụng các từ khóa And và Or để kết hợp các điều kiện tìm kiếm.
- **Tên thuộc tính:** Bạn phải sử dụng tên của các thuộc tính trong Entity để chỉ định điều kiện trong truy vấn.

- Code ví dụ:

```
@Repository 2 usages  🧑 kienn25 *  
public interface ClaimRepository extends JpaRepository<ClaimEntity, Long> {  
  
    public List<ClaimEntity> findByCodeAndClaimDate(String code, LocalDate claimDate);  
}
```

5. Query Creation

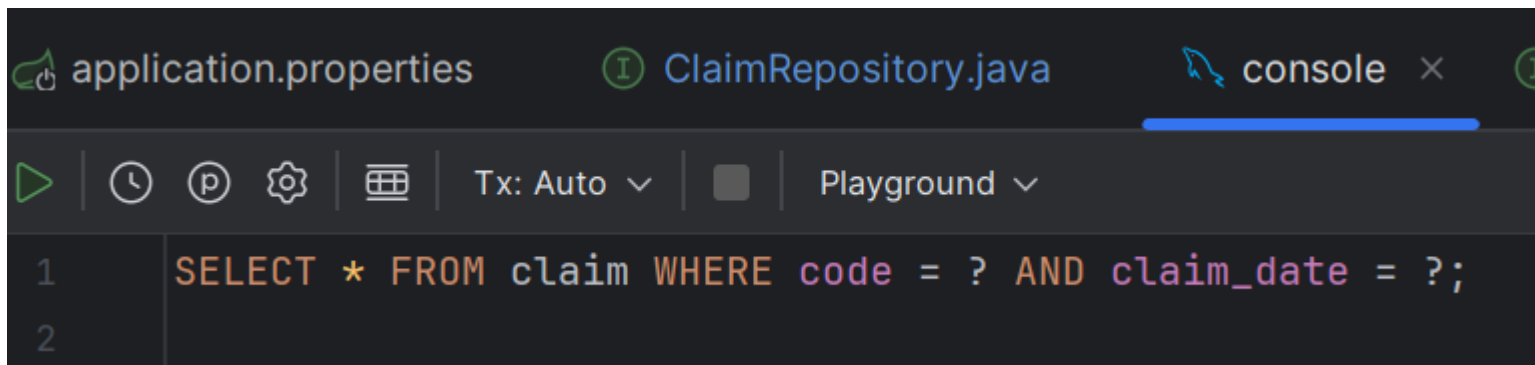
Phân tích ví dụ tên phương thức `findByCodeAndClaimDate`:

- **findBy:** Đây là từ khóa chỉ hành động tìm kiếm, báo hiệu cho Spring Data JPA rằng bạn muốn thực hiện một truy vấn tìm kiếm.
- **Code:** Đây là tên thuộc tính của Entity `ClaimEntity`. Spring Data JPA sẽ tự động tạo ra một điều kiện để tìm kiếm các bản ghi có giá trị code phù hợp.
- **And:** Từ khóa này cho biết rằng bạn muốn kết hợp hai điều kiện (logical AND) trong truy vấn. Điều này có nghĩa là cả hai điều kiện sau đây phải đúng.
- **ClaimDate:** Đây là tên thuộc tính thứ hai của Entity `ClaimEntity`. Spring Data JPA sẽ tạo một điều kiện tìm kiếm cho trường `claimDate`.

```
@Repository 2 usages  • kiennd25 *  
public interface ClaimRepository extends JpaRepository<ClaimEntity, Long> {  
  
    public List<ClaimEntity> findByCodeAndClaimDate(String code, LocalDate claimDate);  
}
```

5. Query Creation

- Spring Data JPA sẽ tự động tạo ra câu truy vấn SQL hoặc JPQL tương ứng
- **SQL:** Giả sử bạn đang sử dụng Hibernate (một implementation của JPA), câu truy vấn có thể giống như sau:

A screenshot of an IDE's console window. The title bar shows three tabs: 'application.properties', 'ClaimRepository.java', and 'console'. The 'console' tab is active and highlighted with a blue underline. Below the tabs is a toolbar with icons for running, debugging, and other IDE functions, along with 'Tx: Auto' and 'Playground' dropdowns. The console area displays a SQL query on two lines: '1 SELECT * FROM claim WHERE code = ? AND claim_date = ?;' and '2'.

```
1 SELECT * FROM claim WHERE code = ? AND claim_date = ?;  
2
```

- Ở đây, code và claim_date là các cột trong bảng claim ứng với entity ClaimEntity

5. Query Creation

- Các ví dụ khác

@Repository 2 usages · kiennd25 *

```
public interface ClaimRepository extends JpaRepository<ClaimEntity, Long> {
```

```
    List<ClaimEntity> findByCustomerEntity(CustomerEntity customerEntity); no usages new *
```

```
    List<ClaimEntity> findByClaimStatusEntity(ClaimStatusEntity claimStatusEntity); no usages new *
```

```
    public List<ClaimEntity> findByCodeAndClaimDate(String code, LocalDate claimDate); no usages new *
```

6) Các annotation phổ biến

- Các annotation phổ biến trong Spring Data JPA giúp bạn tùy chỉnh cách thức thực hiện các truy vấn và xử lý giao dịch. Dưới đây là giải thích chi tiết về các annotation
 1. @Query,
 2. @Modifying
 3. @Transactional
- Ví dụ cùng với ClaimEntity.

6.1) @Query

- **@Query** là một annotation trong Spring Data JPA dùng để viết các truy vấn tùy chỉnh bằng JPQL (Java Persistence Query Language) hoặc native SQL (SQL thuần túy). Bạn có thể sử dụng @Query để thực hiện các truy vấn phức tạp mà không cần phải dựa vào tên phương thức của Spring Data JPA.
- **Cách sử dụng:**
 - ❑ @Query có thể được sử dụng để viết truy vấn JPQL hoặc native query tùy chỉnh. Bạn có thể định nghĩa câu truy vấn trực tiếp trong annotation.

6.1) @Query

- **Ví dụ:** Giả sử bạn muốn tìm kiếm tất cả các yêu cầu bồi thường (ClaimEntity) của một khách hàng cụ thể dựa trên customer_id:

```
@Repository 2 usages  • kienn25 *  
public interface ClaimRepository extends JpaRepository<ClaimEntity, Long> {  
    // Sử dụng JPQL để tìm tất cả các claim của một customer  
    @Query("SELECT c FROM ClaimEntity c WHERE c.customerEntity.id = :customerId") no usages new *  
    List<ClaimEntity> findClaimsByCustomerId(@Param("customerId") Long customerId);  
}
```

- **Giải thích:**
 - ❑ **@Query:** Viết một truy vấn JPQL để tìm tất cả các yêu cầu bồi thường (ClaimEntity) có customerEntity.id trùng với customerId truyền vào.
 - ❑ **@Param("customerId"):** Tham số customerId được ánh xạ với giá trị mà bạn truyền vào phương thức.

6.2) @Modifying

- **@Modifying** là một annotation được sử dụng để chỉ định rằng truy vấn sẽ thực hiện một thao tác sửa đổi dữ liệu, như UPDATE, DELETE, hoặc INSERT. Phương thức có annotation này thường sẽ không trả về một đối tượng mà chỉ thực hiện thay đổi trên cơ sở dữ liệu.

Cách sử dụng:

- Khi bạn sử dụng @Modifying, bạn cần sử dụng với @Query để thực hiện các truy vấn như UPDATE, DELETE, hoặc bất kỳ thao tác sửa đổi nào trong cơ sở dữ liệu.

6.2) @Modifying

- Ví dụ cập nhật trạng thái của một yêu cầu bồi thường (ClaimEntity):

```
@Repository 2 usages  • kienn25 *  
public interface ClaimRepository extends JpaRepository<ClaimEntity, Long> {  
    // Cập nhật trạng thái của một claim theo code  
    @Transactional no usages new *  
    @Modifying  
    @Query("UPDATE ClaimEntity c SET c.claimStatusEntity = :status WHERE c.code = :code")  
    void updateClaimStatus(@Param("status") ClaimStatusEntity status, @Param("code") String code);  
}
```

- Giải thích:
 - ❑ **@Modifying:** Chỉ ra rằng truy vấn này sẽ thực hiện một thao tác sửa đổi dữ liệu (ở đây là UPDATE).
 - ❑ **@Query:** Viết một truy vấn JPQL để cập nhật trạng thái của ClaimEntity theo code.
 - ❑ **@Param("status")** và **@Param("code"):** Ánh xạ tham số status và code vào giá trị truyền vào phương thức.
 - ❑ **Lưu ý:** Khi sử dụng @Modifying, phương thức cần được thực thi trong một giao dịch (transaction), vì vậy bạn cần sử dụng @Transactional cùng với @Modifying (giống ví dụ dưới đây).

6.3) @Transactional

- @Transactional là một annotation trong Spring giúp đảm bảo rằng các thao tác trong phương thức được thực hiện trong một giao dịch (transaction). Điều này có nghĩa là nếu có lỗi xảy ra trong quá trình thực hiện các thao tác, toàn bộ giao dịch sẽ bị rollback (hủy bỏ), giúp bảo vệ tính toàn vẹn của dữ liệu.
- Cách sử dụng:
 - ❑ @Transactional có thể được sử dụng ở cấp độ phương thức hoặc lớp. Đảm bảo rằng các thao tác dữ liệu trong phương thức như save(), update(), delete() sẽ được thực hiện trong một giao dịch.

6.3) @Transactional

- Ví dụ: Giả sử muốn thực hiện cập nhật trạng thái của một yêu cầu bồi thường và sau đó lưu lại các thay đổi vào cơ sở dữ liệu trong một giao dịch duy nhất:

```
@Repository 2 usages  • kiennd25 *  
public interface ClaimRepository extends JpaRepository<ClaimEntity, Long> {  
    @Modifying no usages new *  
    @Transactional // Đảm bảo phương thức thực hiện trong một giao dịch  
    @Query("UPDATE ClaimEntity c SET c.claimStatusEntity = :status WHERE c.code = :code")  
    void updateClaimStatus(@Param("status") ClaimStatusEntity status, @Param("code") String code);  
}
```

- @Transactional:** Đảm bảo rằng phương thức thực thi trong một giao dịch. Nếu có lỗi trong quá trình thực thi (ví dụ: lỗi cập nhật dữ liệu), tất cả các thay đổi sẽ bị rollback.

7) Native Query trong Spring Data JPA

- Native Query trong Spring Data JPA là cách sử dụng SQL thuần túy để thực hiện các truy vấn trực tiếp vào cơ sở dữ liệu, thay vì sử dụng JPQL (Java Persistence Query Language). Điều này rất hữu ích khi bạn cần sử dụng các câu truy vấn đặc biệt hoặc phức tạp mà JPQL không thể hỗ trợ được.
- Tại sao sử dụng Native Query?
 - ❑ **Hỗ trợ SQL đặc biệt:** Khi bạn cần sử dụng các tính năng của cơ sở dữ liệu cụ thể mà JPQL không hỗ trợ, chẳng hạn như các hàm SQL đặc biệt hoặc cú pháp SQL riêng biệt của từng hệ quản trị cơ sở dữ liệu (DBMS).
 - ❑ **Hiệu suất:** Native Query có thể tối ưu hơn trong một số trường hợp, vì bạn có thể viết trực tiếp câu truy vấn SQL mà không cần qua các bước trung gian của JPA.
 - ❑ **Các truy vấn phức tạp:** Nếu bạn cần thực hiện các phép toán phức tạp hoặc các phép kết nối nhiều bảng mà JPQL không dễ dàng mô tả.

7) Native Query trong Spring Data JPA

Cách sử dụng Native Query trong Spring Data JPA

- Spring Data JPA hỗ trợ việc sử dụng native SQL thông qua annotation `@Query` với thuộc tính `nativeQuery = true`. Khi sử dụng `nativeQuery`, Spring Data JPA sẽ sử dụng SQL thuần túy thay vì JPQL.
- **Cấu trúc câu lệnh sử dụng Native Query:**
 - ❑ `@Query(value = "YOUR_NATIVE_SQL_QUERY", nativeQuery = true)`
 - ❑ `value`: Đây là câu truy vấn SQL thuần túy bạn muốn sử dụng.
 - ❑ `nativeQuery = true`: Chỉ định rằng truy vấn này là một native SQL

7) Native Query trong Spring Data JPA

Ví dụ về Native Query với Entity ClaimEntity

- Giả sử bạn muốn tìm tất cả các yêu cầu bồi thường (ClaimEntity) có claimDate nằm trong khoảng thời gian nhất định và có trạng thái cụ thể, sử dụng native SQL thay vì JPQL.

```
@Repository 2 usages  • kienn25 *
public interface ClaimRepository extends JpaRepository<ClaimEntity, Long> {

    // Sử dụng native query để tìm các claim trong một khoảng thời gian cụ thể với trạng thái
    @Query(value = "SELECT * FROM claim c " + no usages  new *
        "WHERE c.claim_date BETWEEN :startDate AND :endDate " +
        "AND c.status_id = :statusId " +
        "AND (c.deleted = false OR c.deleted IS NULL)",
        nativeQuery = true)
    List<ClaimEntity> findClaimsByDateRangeAndStatus(@Param("startDate") LocalDate startDate,
        @Param("endDate") LocalDate endDate,
        @Param("statusId") Long statusId);
```

7) Native Query trong Spring Data JPA

Giải thích:

- ❑ @Query: Được sử dụng để viết một câu truy vấn SQL thuần túy.
- ❑ nativeQuery = true: Đảm bảo rằng câu truy vấn này sử dụng SQL gốc thay vì JPQL.
- ❑ SELECT * FROM claim c: Truy vấn này tìm tất cả các yêu cầu bồi thường (ClaimEntity) từ bảng claim.
- ❑ WHERE c.claim_date BETWEEN :startDate AND :endDate: Điều kiện tìm kiếm yêu cầu bồi thường trong khoảng thời gian giữa startDate và endDate.
- ❑ AND c.status_id = :statusId: Điều kiện tìm kiếm yêu cầu có trạng thái (status_id) trùng với statusId.
- ❑ AND (c.deleted = false OR c.deleted IS NULL): Điều kiện đảm bảo rằng các yêu cầu chưa bị xóa hoặc trường deleted có giá trị NULL.

7) Native Query trong Spring Data JPA

Sự khác biệt giữa JPQL và Native Query:

- JPQL (Java Persistence Query Language):
 - ☐ Là một ngôn ngữ truy vấn đối tượng, không phụ thuộc vào cơ sở dữ liệu cụ thể.
 - ☐ JPQL sử dụng tên đối tượng (Entity) và tên thuộc tính thay vì tên bảng và cột trong cơ sở dữ liệu.
- Native Query:
 - ☐ Là SQL thuần túy, bạn có thể viết truy vấn trực tiếp bằng SQL và truy cập vào cơ sở dữ liệu thông qua các câu lệnh SQL cụ thể cho từng DBMS (Oracle, MySQL, PostgreSQL...).

7) Native Query trong Spring Data JPA

Native Query với Kết quả Tùy chỉnh (DTO Projection)

- Đôi khi bạn không muốn lấy toàn bộ Entity mà chỉ muốn một số cột nhất định từ cơ sở dữ liệu. Bạn có thể sử dụng DTO Projection để trả về các kết quả dưới dạng một đối tượng cụ thể.
- **Ví dụ:** nếu bạn chỉ cần trả về claimCode và claimDate, bạn có thể viết như sau:

```
public class ClaimProjectionDTO { no usages new *  
    private String claimCode; 2 usages  
    private LocalDate claimDate; 2 usages  
  
    public String getClaimCode() { no usages new *  
        return claimCode;  
    }  
  
    public void setClaimCode(String claimCode) { no usages new *  
        this.claimCode = claimCode;  
    }  
}
```

7) Native Query trong Spring Data JPA

Code trong repository

```
@Repository 2 usages  • kiennd25 *  
public interface ClaimRepository extends JpaRepository<ClaimEntity, Long> {  
  
    @Query(value = "SELECT c.code AS claimCode, c.claim_date AS claimDate FROM claim c " +  
        "WHERE c.claim_date BETWEEN :startDate AND :endDate " +  
        "AND c.status_id = :statusId " +  
        "AND (c.deleted = false OR c.deleted IS NULL)",  
        nativeQuery = true)  
    List<ClaimProjectionDTO> findClaimsDTOByDateRangeAndStatus(@Param("startDate") LocalDate startDate,  
                                                                @Param("endDate") LocalDate endDate,  
                                                                @Param("statusId") Long statusId);  
}
```

Giải thích:

- ❑ DTO Projection: Trong câu truy vấn native SQL, bạn chỉ chọn các cột bạn cần (ví dụ c.code và c.claim_date) và ánh xạ chúng vào đối tượng DTO (ClaimProjectionDTO).
- ❑ @Query với nativeQuery = true: Thực thi câu truy vấn SQL thuần túy và ánh xạ kết quả vào đối tượng ClaimDTO.

7) Native Query trong Spring Data JPA

Lợi ích của Native Query trong Spring Data JPA:

- **Sử dụng SQL thuần túy:** Khi bạn cần sử dụng tính năng đặc biệt của cơ sở dữ liệu hoặc tối ưu hóa các truy vấn.
- **Hỗ trợ các câu truy vấn phức tạp:** Các câu truy vấn yêu cầu cú pháp SQL đặc biệt hoặc không thể thực hiện được với JPQL.
- **Hiệu suất:** Trong một số trường hợp, câu truy vấn SQL thuần túy có thể nhanh hơn JPQL, đặc biệt khi cần thực hiện các phép toán phức tạp.

